Практичне заняття № 4

Тема: «Робота з інтерфейсами. Реалізація множинної поведінки класів у Java»

Мета заняття: ознайомитися з поняттям інтерфейсу в Java; навчитися оголошувати інтерфейси та реалізовувати їх за допомогою implements; реалізовувати кілька інтерфейсів одним класом; поєднувати успадкування класів та інтерфейси; використовувати посилання інтерфейсного типу, default- і static-методи; закріпити знання про класи, об'єкти, конструктори, extends, super, enum та перевантаження методів.

Короткі теоретичні положення

Інтерфейс (interface) задає набір можливостей, які клас зобов'язується реалізувати. Інтерфейс описує, що об'єкт уміє робити, але звичайно не зберігає стан конкретного об'єкта.

Оголошення інтерфейсу. Для оголошення використовується ключове слово interface. Методи без реалізації задають контракт для класів, які реалізують цей інтерфейс.

implements. Клас реалізує інтерфейс за допомогою implements. Якщо клас реалізує інтерфейс, він повинен надати реалізації його абстрактних методів або сам бути abstract.

public у реалізації. Методи інтерфейсу без явного модифікатора є public abstract. Тому реалізований у класі метод також має бути public; звужувати доступ не можна.

Кілька інтерфейсів. Один клас може реалізувати кілька інтерфейсів. Їх назви перелічують через кому: class Robot implements Movable, Chargeable.

extends і implements. extends описує успадкування від класу, а implements – реалізацію інтерфейсу. Клас може одночасно успадковувати один клас і реалізовувати кілька інтерфейсів.

Інтерфейсний тип. Змінна типу інтерфейсу може посилатися на об'єкт будь-якого класу, який реалізує цей інтерфейс. Це один із важливих кроків до поліморфізму.

Константи. Поля, оголошені в інтерфейсі, фактично є public static final. Їх потрібно одразу ініціалізувати.

default-методи. Метод із ключовим словом default може мати реалізацію безпосередньо в інтерфейсі. Клас отримує цю реалізацію та за потреби може її перевизначити.

static-методи. Інтерфейс може містити static-методи. Вони викликаються через ім'я інтерфейсу, наприклад Printable.info().

Приклад 1. Найпростіший інтерфейс

```
interface Printable {
    void print();
}

class Document implements Printable {
    public void print() {
        System.out.println("Документ надруковано");
    }
}
```

Printable задає вимогу мати метод print(). Клас Document реалізує цю вимогу. Зверніть увагу: реалізація print() оголошена public.

Приклад 2. Інтерфейс із параметрами

```
interface Movable {
    void move(int distance);
}

class Robot implements Movable {
    public void move(int distance) {
        System.out.println("Робот проїхав " + distance + "
м");
    }
}
```

Приклад 3. Один клас реалізує два інтерфейси

```
interface Printable {
    void print();
}

interface Downloadable {
    void download();
}

class Document implements Printable, Downloadable {
    String name;
```

```
    Document(String name) {
        this.name = name;
    }

    public void print() {
        System.out.println("Друк: " + name);
    }

    public void download() {
        System.out.println("Завантаження: " + name);
    }
}
```

Приклад 4. extends + implements

```
class Person {
    String name;

    Person(String name) {
        this.name = name;
    }
}

interface Printable {
    void print();
}

interface Loginable {
    void login();
}

class Student extends Person implements Printable,
Loginable {
    Student(String name) {
        super(name);
    }

    public void print() {
        System.out.println("Студент: " + name);
```

```
    }

    public void login() {
        System.out.println(name + " увійшов у систему");
    }
}
```

Student є різновидом Person, тому використовується extends. Водночас Student має можливості Printable і Loginable, тому використовується implements.

Приклад 5. Посилання інтерфейсного типу

```
Printable item = new Document("Report.pdf");
item.print();
```

Тип змінної – Printable, а реальний об’єкт – Document. Через таке посилання доступні методи, визначені інтерфейсом Printable.

Приклад 6. Масив інтерфейсного типу

```
interface Printable {
    void print();
}

class Report implements Printable {
    public void print() {
        System.out.println("Друк звіту");
    }
}

class Photo implements Printable {
    public void print() {
        System.out.println("Друк фотографії");
    }
}

public class Main {
    public static void main(String[] args) {
        Printable[] items = {
            new Report(),
            new Photo()
```

```
        };

        for (Printable item : items) {
            item.print();
        }
    }
}
```

В одному масиві можна зберігати об’єкти різних класів, якщо всі вони реалізують Printable.

Приклад 7. default-метод

```
interface Logger {
    void log(String message);

    default void showStatus() {
        System.out.println("Logger працює");
    }
}

class FileLogger implements Logger {
    public void log(String message) {
        System.out.println("LOG: " + message);
    }
}
```

Приклад 8. static-метод інтерфейсу

```
interface Calculator {
    static int square(int x) {
        return x * x;
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println(Calculator.square(5));
    }
}
```

Приклад 9. Константа в інтерфейсі

```
interface Config {
    int MAX_USERS = 100;
}

System.out.println(Config.MAX_USERS);
```

MAX_USERS фактично є public static final, тому змінити її значення не можна.

extends чи implements?

Ознака	extends	implements
Що пов'язує?	клас із базовим класом	клас з інтерфейсом
Основна ідея	«є різновидом»	«уміє виконувати»
Скільки для класу?	один базовий клас	кілька інтерфейсів
Приклад	Student extends Person	Student implements Printable

Типові помилки

- Написати extends замість implements для реалізації інтерфейсу.
- Забути public перед методом, який реалізує метод інтерфейсу.
- Не реалізувати один із обов'язкових методів інтерфейсу.
- Спробувати створити об'єкт самого інтерфейсу: new Printable() – так робити не можна.
- Плутати інтерфейс із класом: інтерфейс задає контракт поведінки, а клас створює конкретні об'єкти.
- Викликати static-метод інтерфейсу через об'єкт замість імені інтерфейсу.

Повний приклад. Робот

```
enum RobotType {
    INDUSTRIAL, DELIVERY, SERVICE
}

class Device {
    String model;

    Device(String model) {
        this.model = model;
    }
}
```

```
interface Movable {
    void move(int distance);
    void stop();
}

interface Rechargeable {
    void charge(int percent);
}

class Robot extends Device implements Movable, Rechargeable
{
    RobotType type;

    Robot(String model, RobotType type) {
        super(model);
        this.type = type;
    }

    public void move(int distance) {
        System.out.println(model + " рухається " + distance
+ " м");
    }

    public void stop() {
        System.out.println(model + " зупинився");
    }

    public void charge(int percent) {
        System.out.println("Заряд: " + percent + "%");
    }
}

public class Main {
    public static void main(String[] args) {
        Robot robot = new Robot("R-10",
RobotType.DELIVERY);
        robot.move(50);
        robot.stop();
```

```
        robot.charge(80);
    }
}
```

Хід виконання роботи

За своїм варіантом створити програму, у якій один клас реалізує щонайменше два інтерфейси. Бажано пов'язати її з матеріалом попередньої практичної.

- створити два інтерфейси відповідно до предметної області;
- у кожному інтерфейсі оголосити щонайменше один метод;
- створити клас, який реалізує обидва інтерфейси;
- реалізувати всі обов'язкові методи з модифікатором public;
- додати поля та конструктор класу;
- за можливості використати базовий клас і конструкцію extends + implements;
- створити не менше двох об'єктів;
- продемонструвати роботу всіх реалізованих методів;
- створити хоча б одне посилання інтерфейсного типу;
- для підвищеного рівня – використати масив інтерфейсного типу або default-метод.

Варіанти індивідуальних завдань

Варіант	Мініпроєкт	Ієрархія	Інтерфейси	Завдання
B.1	Студентська система	Person → Student	Printable, Loginable	print() – вивести дані студента; login() – повідомити про вхід у систему.
B.2	Робот	Device → Robot	Movable, Rechargeable	move() – рух; charge() – заряджання. Додати stop().
B.3	Смартфон	Device → Smartphone	Callable, PhotoCapable	call() – виконати дзвінок; takePhoto() – зробити фото.
B.4	Електромобіль	Vehicle → ElectricCar	Drivable, Rechargeable	drive() – поїздка; charge() – заряджання батареї.
B.5	Принтер	Device → Printer	Printable, Scannable	print() – друк документа; scan() – сканування.

B.6	Дрон	Device → Drone	Flyable, PhotoCapable	fly() – політ на задану висоту; takePhoto() – фотографування.
B.7	Медіаплеєр	Device → MediaPlayer	Playable, Recordable	play() – відтворення; record() – запис.
B.8	Онлайн-курс	Course → OnlineCourse	Downloadable, Certifiable	download() – завантаження матеріалів; issueCertificate() – видача сертифіката.
B.9	Банківський рахунок	Account → BankAccount	Payable, Transferable	pay() – оплата; transfer() – переказ коштів.
B.10	Кур’єрський робот	Robot → DeliveryRobot	Movable, Deliverable	move() – рух; deliver() – доставка замовлення.
B.11	Розумний годинник	Device → SmartWatch	Notifiable, Measurable	notifyUser() – сповіщення; measure() – вимірювання показника.
B.12	Книга в електронній бібліотеці	Publication → EBook	Readable, Downloadable	read() – читання; download() – завантаження.
B.13	Музичний сервіс	Media → Song	Playable, Downloadable	play() – відтворення; download() – завантаження композиції.
B.14	Таксі	Transport → Taxi	Drivable, Payable	drive() – поїздка; pay() – оплата поїздки.
B.15	Працівник	Person → Employee	Workable, Reportable	work() – виконання роботи; createReport() – формування звіту.
B.16	Ігровий персонаж	Character → Player	Movable, Attackable	move() – переміщення; attack() – атака.
B.17	Система повідомлень	Message → EmailMessage	Sendable, Printable	send() – відправлення; print() – друк повідомлення.

B.18	Розумна лампа	Device → SmartLamp	Switchable, Dimmable	turnOn()/turnOff() – увімкнення/вимкнення; setBrightness() – яскравість.
B.19	Файл	FileItem → CloudFile	Downloadable, Shareable	download() – завантаження; share() – надання доступу.
B.20	Квиток на подію	Ticket → ElectronicTicket	Printable, Verifiable	print() – друк квитка; verify() – перевірка квитка.

Завдання «Знайди помилку»

```
interface Printable {
    void print();
}

class Report implements Printable {
    void print() {
        System.out.println("Report");
    }
}
```

Чому код не компілюється? Виправте його та поясніть роль модифікатора public.

Завдання «Що виведе програма?»

```
interface Soundable {
    void sound();
}

class Cat implements Soundable {
    public void sound() {
        System.out.println("Meow");
    }
}

public class Main {
    public static void main(String[] args) {
        Soundable animal = new Cat();
        animal.sound();
```

```
        }
    }
```

Запишіть відповідь до запуску та поясніть, чому змінна типу Soundable може посилатися на Cat.

Додаткове завдання підвищеної складності

Створіть масив інтерфейсного типу, у якому містяться об’єкти щонайменше двох різних класів, що реалізують один інтерфейс. За допомогою циклу викличте спільний метод для кожного елемента.

До звіту необхідно включити

- умову індивідуального завдання;
- схему класів та інтерфейсів;
- лістинг усіх класів та інтерфейсів;
- пояснення використання extends та implements;
- пояснення, які методи оголошені в інтерфейсах і де вони реалізовані;
- приклад використання посилання інтерфейсного типу;
- скріншот результату виконання програми;
- висновки.

Контрольні питання

1. Що таке інтерфейс у Java?
2. Яке ключове слово використовується для оголошення інтерфейсу?
3. Для чого використовується implements?
4. Чим extends відрізняється від implements?
5. Чи може клас реалізувати кілька інтерфейсів?
6. Як записати реалізацію двох інтерфейсів одним класом?
7. Чому метод реалізації інтерфейсу повинен бути public?
8. Чи можна створити об’єкт інтерфейсу за допомогою new?
9. Що станеться, якщо клас не реалізує всі обов’язкові методи інтерфейсу?
10. Чи може клас одночасно успадковувати клас і реалізовувати інтерфейси?
11. Що таке посилання інтерфейсного типу?
12. Які об’єкти можна присвоїти змінній інтерфейсного типу?
13. Що таке default-метод?
14. Чи має default-метод реалізацію?
15. Що таке static-метод інтерфейсу?
16. Як викликається static-метод інтерфейсу?
17. Якими фактично є поля інтерфейсу?
18. Чи можна змінити значення константи інтерфейсу?
19. Для чого може бути корисним масив інтерфейсного типу?

20. Наведіть власний приклад класу, який реалізує два інтерфейси.